Práctica 4

Manuel Suano Gallardo

Ejercicio 1

Para el despliegue de los nodos de Cassandra he utilizado Docker en Ubuntu mediante WSL. Para ello he pasado el archivo `docker-compose.yml` al directorio correspondiente de Ubuntu. Una vez en dicho directorio he ejecutado el comando indicado en la práctica: `docker-compose up -d`, junto con `sudo`, pues me indicaba que no disponía de los permisos necesarios.

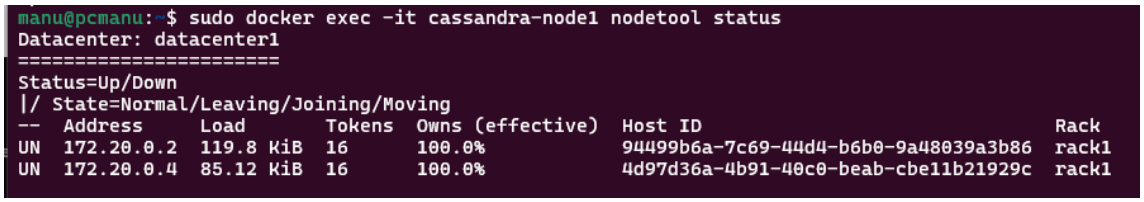
```
manu@pcmanu:~/GAINE/practica4$ sudo docker-compose up -d
[sudo] password for manu:
Sorry, try again.
[sudo] password for manu:
Creating network "practica4_default" with the default driver
Creating volume "practica4_node1_data" with default driver
Creating volume "practica4_node2_data" with default driver
Creating volume "practica4_node3_data" with default driver
Pulling cassandra-node1 (cassandra:latest)...
latest: Pulling from library/cassandra
ff86ea2e5edc: Pull complete
724eef794f35: Pull complete
60f560b8af2e: Pull complete
cc381252655e: Pull complete
9ce54635a440: Pull complete
ad4a04240116: Pull complete
25b691bdeae0: Pull complete
51ca5eb71528: Pull complete
Digest: sha256:0e4d8e9cafdb9b03e9b0f754dcc6266c67334b7ea4d99eacfcd0711438a8a6f5
Status: Downloaded newer image for cassandra:latest
Creating cassandra-node1 ... done
Creating cassandra-node2 ... done
Creating cassandra-node3 ... done
manu@pcmanu:~/GAINE/practica4$ ls
docker-compose.yml  docker-compose.yml:Zone.Identifier
manu@pcmanu:~/GAINE/practica4$
```

Una vez ejecutado el comando, podemos comprobar que los nodos están desplegados (en dos imágenes para que se pueda observar correctamente):

```
manu@pcmanu:~$ sudo docker ps
[sudo] password for manu:
CONTAINER ID   IMAGE              COMMAND                  CREATED          STATUS
1d4c1d515f44   cassandra:latest   "docker-entrypoint.s…"   14 minutes ago   Up 14 minutes
f6cc9f3b542f   cassandra:latest   "docker-entrypoint.s…"   14 minutes ago   Up 14 minutes
07b0141935c1   cassandra:latest   "docker-entrypoint.s…"   15 minutes ago   Up 15 minutes (healthy)
manu@pcmanu:~$
```

```
PORTS                                                                                  NAMES
7000-7001/tcp, 7199/tcp, 9160/tcp, 0.0.0.0:9044->9042/tcp, [::]:9044->9042/tcp   cassandra-node3
7000-7001/tcp, 7199/tcp, 9160/tcp, 0.0.0.0:9043->9042/tcp, [::]:9043->9042/tcp   cassandra-node2
7000-7001/tcp, 7199/tcp, 9160/tcp, 0.0.0.0:9042->9042/tcp, [::]:9042->9042/tcp   cassandra-node1
```

Pasado el tiempo indicado, comprobamos el estado del anillo de Cassandra:



Podemos ver que solo se reconocen dos nodos de los tres que deberían aparecer. Mirando los logs del nodo 2 aparece el error:

> Bootstrap Token collision between /172.20.0.3:7000 and /172.20.0.4:7000

Sobre este caso en concreto no aparece nada en la práctica, por lo que, investigando por cuenta propia y usando herramientas de IA, el error parece que se debe a que hay dos nodos que están intentando conectarse a la vez, en concreto los nodos 2 y 3 están intentando conectarse a la vez al nodo 1 (según el archivo `docker-compose.yml`), de ahí que se produzca el fallo. Para ello modificaré el `docker-compose.yml` para hacer que la conexión del nodo 3 posterior a la conexión del nodo 2, añadiendo un chequeo del estado de este último:

```
cassandra-node2:
# ...
     # Se añade lo mismo que en el nodo 1
     healthcheck:
          test: ["CMD", "cqlsh", "-e", "describe
     keyspaces"]
          interval: 15s
          timeout: 10s
          retries: 10
# ...
cassandra-node3:
# ...
     # Se indica que espere a que el nodo 2 esté listo
     depends_on:
          cassandra-node2:
               condition: service_healthy
```

Para que estos cambios se guarden primero hay que eliminar todos los datos que se tenían hasta ahora con `docker-compose down -v` para asegurarnos que se parte de cero y posteriormente volver a ejecutar `docker-compose up -d`. Ahora sí disponemos de los tres nodos conectados:

```
manu@pcmanu:~$ sudo docker exec -it cassandra-node1 nodetool status
[sudo] password for manu:
Datacenter: datacenter1
=======================
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
--  Address     Load       Tokens  Owns (effective)  Host ID                               Rack
UN  172.20.0.3  80.04 KiB  16      59.3%             aea2b7d5-2703-49fd-bb35-616a0d7bc566  rack1
UN  172.20.0.2  119.84 KiB 16      64.7%             def0835d-41d5-4809-a90a-44086c8ff311  rack1
UN  172.20.0.4  80.06 KiB  16      76.0%             8969c9fc-49c5-49fb-8a4c-bb884afc0759  rack1
```

Pudiendo ver que se encuentran listos, comprobamos los volúmenes:

```
manu@pcmanu:~$ sudo docker volume ls
DRIVER    VOLUME NAME
local     practica2_redis_data
local     practica4_node1_data
local     practica4_node2_data
local     practica4_node3_data
```

Pudiendo ver los de cada nodo (y el de Redis de la práctica 2).

Continuando con el apartado 2, empezamos ejecutando el comando para ejecutar desde la consola código CQL:

```
manu@pcmanu:~$ sudo docker exec -it cassandra-node1 cqlsh
[sudo] password for manu:

ATTENTION: All commands will be saved to history file: /root/.cassandra/cqlsh_history
This may include sensitive information such as passwords.
To disable history, use --disable-history or set 'disabled = true' in the [history] section of cqlshrc.
See https://cassandra.apache.org/doc/latest/tools/cqlsh.html for more information.

Connected to ClusterClaseDB at 127.0.0.1:9042
[cqlsh 6.2.0 | Cassandra 5.0.8 | CQL spec 3.4.7 | Native protocol v5]
Use HELP for help.
cqlsh>
```

Dentro de la base de datos ejecutamos cada uno de los comandos indicados:

```
cqlsh> CREATE KEYSPACE laboratorio WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 3};
cqlsh> USE laboratorio;
cqlsh:laboratorio> CREATE TABLE alumnos (matricula TEXT PRIMARY KEY, nombre TEXT, curso TEXT);
cqlsh:laboratorio> INSERT INTO alumnos (matricula, nombre, curso) VALUES ('A001', 'Ada Lovelace', 'Bases de Datos Avanzadas');
cqlsh:laboratorio> INSERT INTO alumnos (matricula, nombre, curso) VALUES ('A002', 'Alan Turing', 'Sistemas Distribuidos');
cqlsh:laboratorio> SELECT * FROM alumnos;

 matricula | curso                    | nombre
-----------+--------------------------+--------------
      A001 | Bases de Datos Avanzadas | Ada Lovelace
      A002 |     Sistemas Distribuidos |  Alan Turing

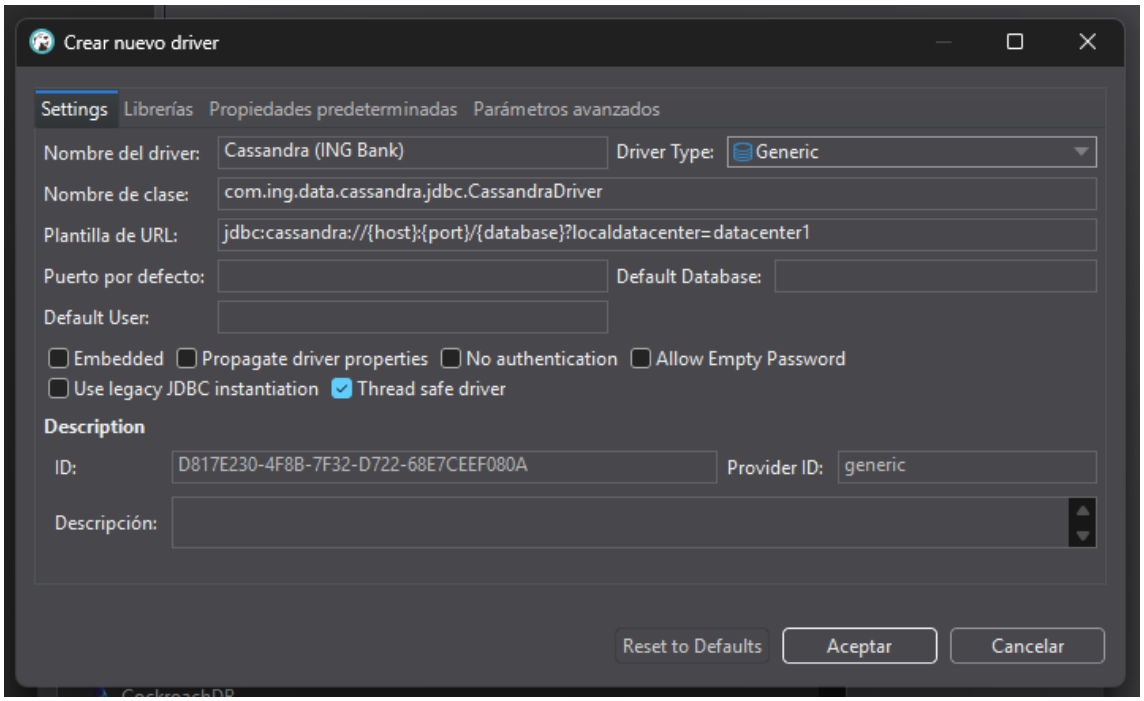
(2 rows)
cqlsh:laboratorio> SELECT * FROM alumnos WHERE matricula = 'A001';

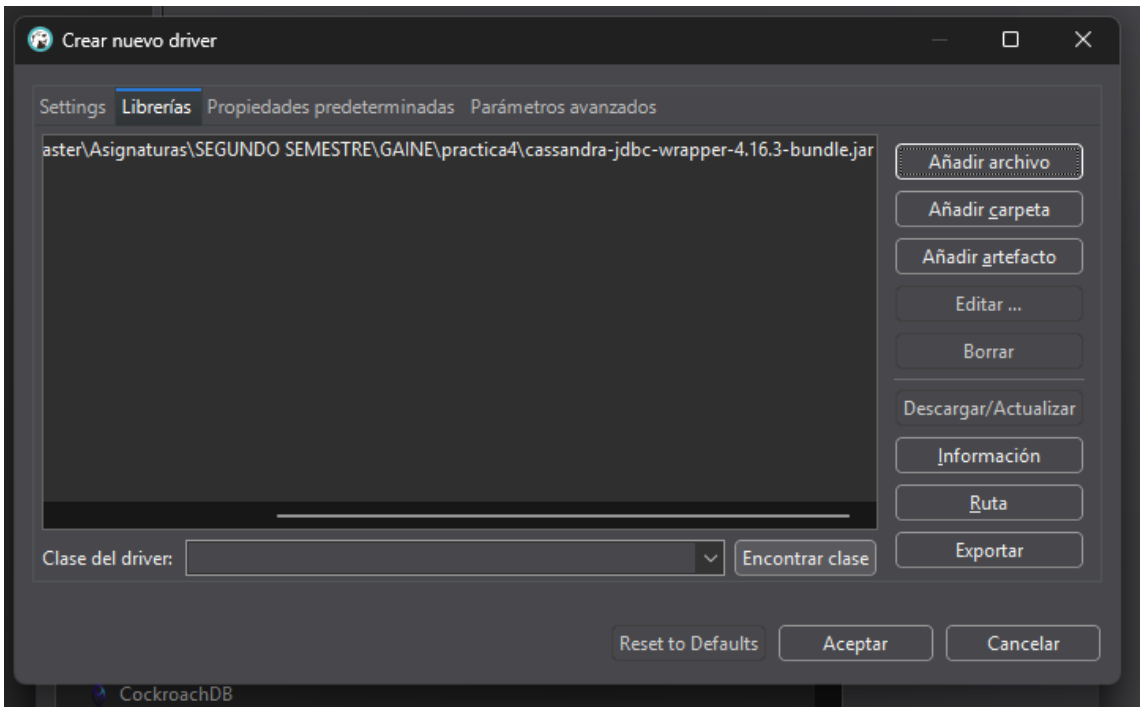
 matricula | curso                    | nombre
-----------+--------------------------+--------------
      A001 | Bases de Datos Avanzadas | Ada Lovelace

(1 rows)
cqlsh:laboratorio> exit
manu@pcmanu:~$
```

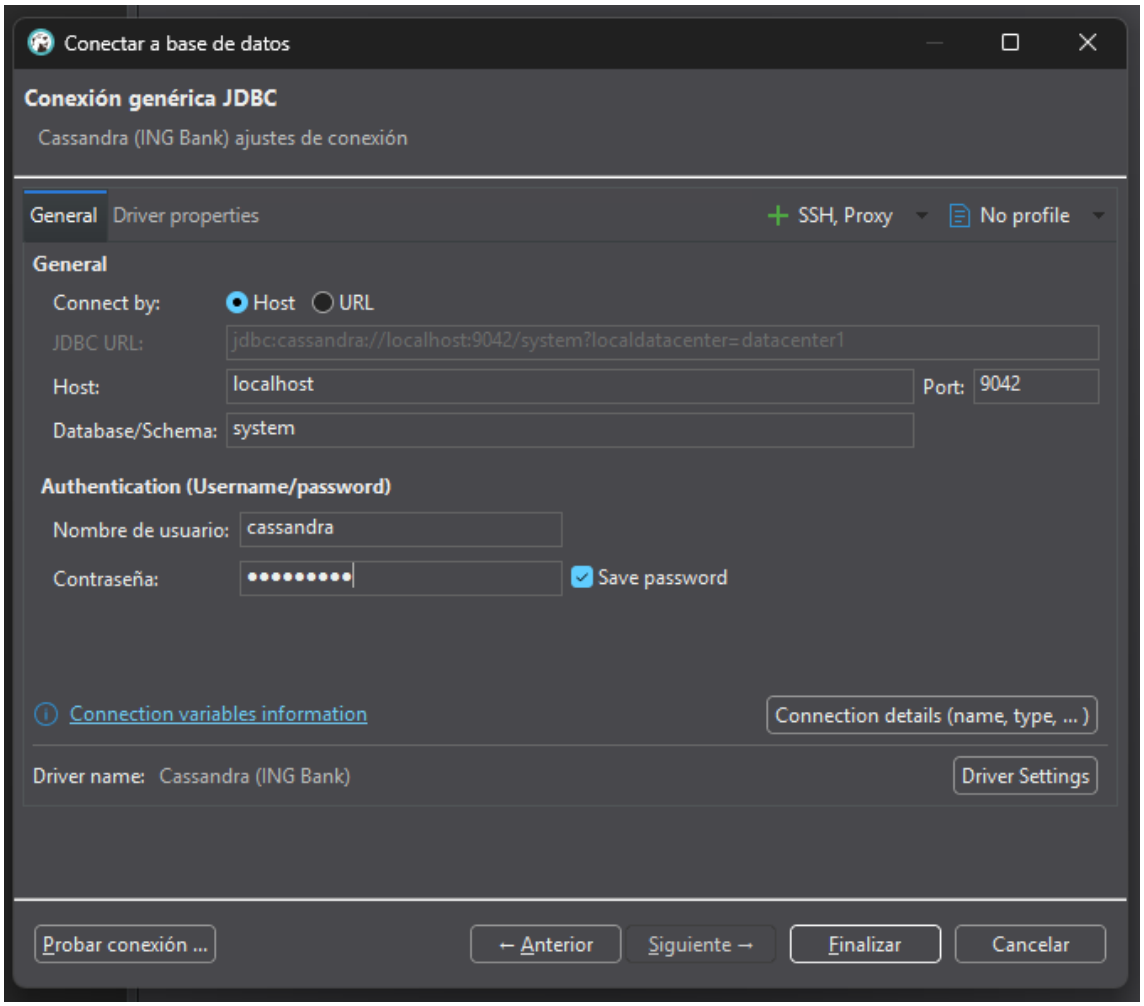
Pudiendo comprobar que todos se han ejecutado correctamente y se obtiene las salidas esperadas.

Por último, pasamos al apartado 3, la conexión con Cassandra mediante DBeaver. Para ello seguimos los pasos indicados en la configuración del controlador, rellenando todos los datos.

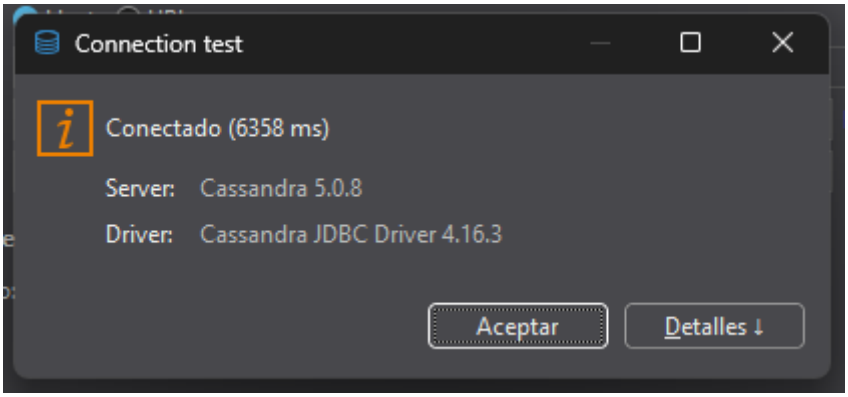

Crear nuevo driver
Settings
Librerías
Propiedades predeterminadas
Parámetros avanzados
Nombre del driver:
Cassandra (ING Bank)
Driver Type:
Generic
Nombre de clase:
com.ing.data.cassandra.jdbc.CassandraDriver
Plantilla de URL:
jdbc:cassandra://{host}:{port}/{database}?localdatacenter=datacenter1
Puerto por defecto:
Default Database:
Default User:
Embedded
Propagate driver properties
No authentication
Allow Empty Password
Use legacy JDBC instantiation
Thread safe driver
Description
ID:
D817E230-4F8B-7F32-D722-68E7CEEF080A
Provider ID:
generic
Descripción:
Reset to Defaults
Aceptar
Cancelar


Crear nuevo driver
Settings
Librerías
Propiedades predeterminadas
Parámetros avanzados
aster\Asignaturas\SEGUNDO SEMESTRE\GAINE\practica4\cassandra-jdbc-wrapper-4.16.3-bundle.jar
Añadir archivo
Añadir carpeta
Añadir artefacto
Editar ...
Borrar
Descargar/Actualizar
Información
Ruta
Exportar
Clase del driver:
Encontrar clase
Reset to Defaults
Aceptar
Cancelar
CockroachDB

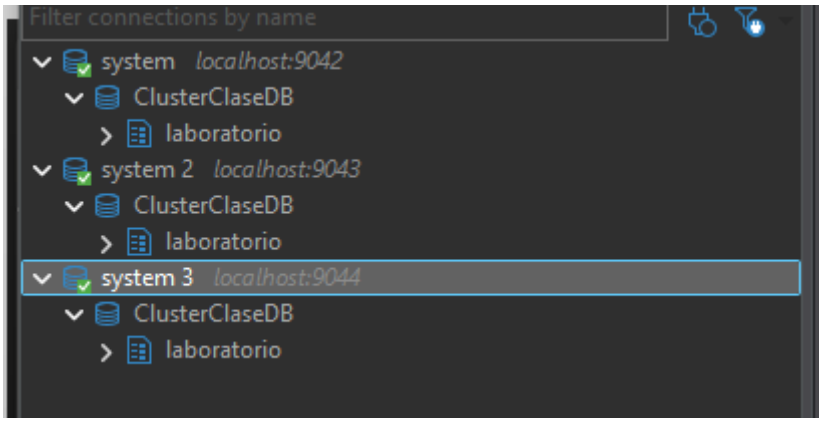
Con esto pasamos a crear la conexión:



Comprobando que se ha conectado correctamente.



Y quedando, finalmente, junto con los otros dos nodos:



Ejercicio 2

Apartado 1.

Primero creamos el keyspace. Para ello ejecutaremos el siguiente comando:

```
CREATE KEYSPACE IF NOT EXISTS logistica_flotas
WITH REPLICATION = {
      'class' : 'NetworkTopologyStrategy',
      'datacenter1' : 3
};
```

He optado por usar la clase `NetworkTopologyStrategy` debido a que, según se indica en la web indicada en las diapositivas de CQL[1], está destinada a entornos de producción. Como nombre del centro de datos se indica `datacenter1` ya que es el que por defecto se ha establecido (se puede comprobar al ejecutar los comandos `docker exec -it cassandra-node<1,2,3> nodetool status`) y se indica como valor 3 para definir la replicación en 3, una réplica por nodo.

```
cqlsh> CREATE KEYSPACE IF NOT EXISTS logistica_flotas
   ... WITH REPLICATION = {
   ... 'class' : 'NetworkTopologyStrategy',
   ... 'datacenter1' : 3
   ... };
cqlsh>
```

Para almacenar la información indicada y poder realizar las dos consultas indicadas, siguiendo el principio de la desnormalización[2] que se aplica en Cassandra, tendremos dos tablas diferentes, una enfocada a cada consulta.

En el caso de la primera consulta, se tendrá la siguiente tabla:

```
CREATE TABLE logistica_flotas.info_rutas_por_conductor (
```

[1] Para la versión 3.3 en lugar de la 3.1. https://docs.datastax.com/en/cql-oss/3.3/cql/cql_reference/cqlCreateKeyspace.html

[2] https://docs.datastax.com/en/cql/hcd/data-modeling/best-practices.html#denormalization

```
        idConductor text,
        idRuta int,
        fechaRuta date,
        almacenOrigen text,
        almacenDestino text,
        horasEstimadas int,
        PRIMARY KEY (idConductor, idRuta)
    );
```

```
cqlsh:logistica_flotas> CREATE TABLE logistica_flotas.info_rutas_por_conductor (
enDestino text,
horasEstimadas int,
PRIMARY KEY                    ... idConductor text,
Ruta)
);
                   ... idRuta int,
                   ... fechaRuta date,
                   ... almacenOrigen text,
                   ... almacenDestino text,
                   ... horasEstimadas int,
                   ... PRIMARY KEY (idConductor, idRuta)
                   ... );
cqlsh:logistica_flotas>
```

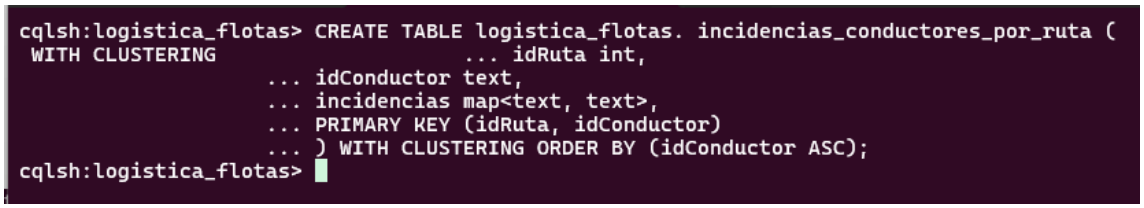
Se define como clave primaria el conjunto de claves `idConductor` (clave de partición) y `idRuta` (clave de ordenamiento, a pesar de que no se necesite para ello). Esta definición permite:

- Realizar consultas indicando un ID de conductor y obtener todas las rutas que ha hecho de forma eficiente, pues se encuentran en la misma partición al ser `idConductor` la clave de partición.
- Conseguir la unicidad de los pares `idConductor` y `idRuta`.

A pesar de que en la consulta se usa la columna `fechaRuta` para aplicar orden a la salida, no sería adecuado introducirla como clave de ordenamiento junto con `idRuta`, pues se permitiría la introducción de un mismo par `idConductor` e `idRuta` pero con distinta fecha, rompiendo la condición de unicidad en la tabla.

Por otro lado, para la segunda consulta, tendremos la tabla:

```
    CREATE TABLE logistica_flotas.
    incidencias_conductores_por_ruta (
        idRuta int,
        idConductor text,
        incidencias map<text, text>,
        PRIMARY KEY (idRuta, idConductor)
    ) WITH CLUSTERING ORDER BY (idConductor ASC);
```



En este caso la clave primaria es de nuevo las columnas `idRuta` e `idConductor`, pero con la primera como clave de partición y la segunda como clave de ordenación. De esta forma conseguimos:

- Realizar consultas indicando un ID de ruta y obtener todos los conductores que la han realizado junto con las incidencias. De nuevo, al usar la columna `idRuta` como clave de partición conseguimos una mejora del rendimiento a la hora de obtener los resultados, pues están todos en la misma partición.
- Unicidad en los pares `idRuta` e `idConductor`.
- Mayor velocidad de lectura, pues los datos ya se encuentran ordenados por la coluna `idConductor` de forma ascendente usando `WITH CLUSTERING ORDER BY`.
- Almacenar varias incidencias diferentes para un mismo par `idRuta` e `idConductor` usando el mapa.

Apartado 2.

Para cada uno de los casos habrá que realizar dos inserciones, uno en cada tabla:

1. ```
   INSERT INTO logistica_flotas.info_rutas_por_conductor
        (idConductor, idRuta, fechaRuta, almacenOrigen,
   almacenDestino, horasEstimadas)
   VALUES
        ('Marcos87', 101, '2024-04-15', 'Madrid', 'Paris',
   5);

   INSERT INTO
   logistica_flotas.incidencias_conductores_por_ruta
        (idRuta, idConductor, incidencias)
   VALUES
        (101, 'Marcos87', {'Aduana' : 'Retraso de 2 horas en
   frontera'});
   ```
2. ```
   INSERT INTO logistica_flotas.info_rutas_por_conductor
        (idConductor, idRuta, fechaRuta, almacenOrigen,
   almacenDestino, horasEstimadas)
   VALUES
        ('CarlosR', 101, '2024-04-15', 'Zaragoza', 'Paris',
   5);
   ```

```
INSERT INTO
logistica_flotas.incidencias_conductores_por_ruta
     (idRuta, idConductor, incidencias)
VALUES
     (101, 'CarlosR', {'Aduana' : 'Retraso de 2 horas en
frontera'});
```

3.
```
INSERT INTO logistica_flotas.info_rutas_por_conductor
     (idConductor, idRuta, fechaRuta, almacenOrigen,
almacenDestino, horasEstimadas)
VALUES
     ('EvaM33', 102, '2024-05-10', 'Valencia', 'Lisboa',
8);

INSERT INTO
logistica_flotas.incidencias_conductores_por_ruta
     (idRuta, idConductor, incidencias)
VALUES
     (102, 'EvaM33', {'Avería' : 'Pinchazo en rueda
delantera'});
```

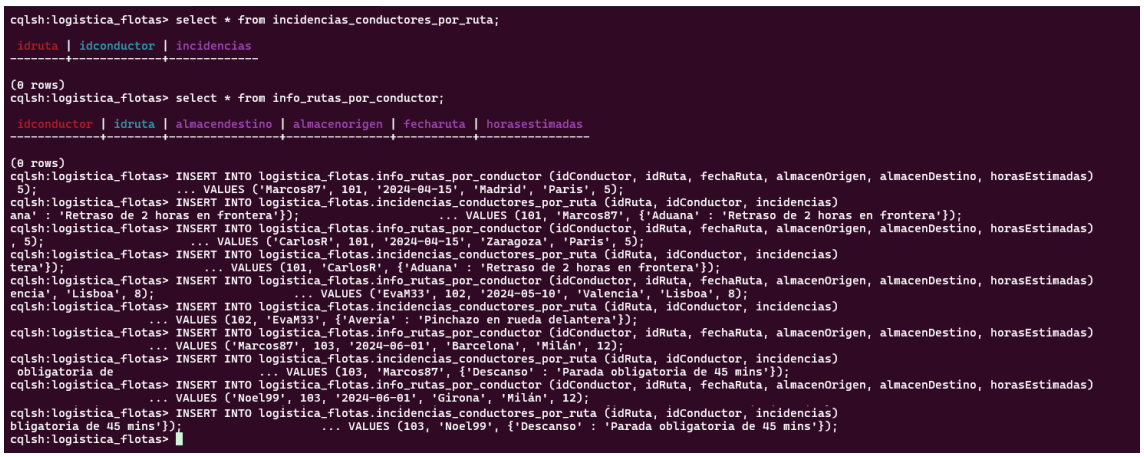
4.
```
INSERT INTO logistica_flotas.info_rutas_por_conductor
     (idConductor, idRuta, fechaRuta, almacenOrigen,
almacenDestino, horasEstimadas)
VALUES
     ('Marcos87', 103, '2024-06-01', 'Barcelona', 'Milán',
12);

INSERT INTO
logistica_flotas.incidencias_conductores_por_ruta
     (idRuta, idConductor, incidencias)
VALUES
     (103, 'Marcos87', {'Descanso' : 'Parada obligatoria
de 45 mins'});
```

5.
```
INSERT INTO logistica_flotas.info_rutas_por_conductor
     (idConductor, idRuta, fechaRuta, almacenOrigen,
almacenDestino, horasEstimadas)
VALUES
     ('Noel99', 103, '2024-06-01', 'Girona', 'Milán', 12);

INSERT INTO
logistica_flotas.incidencias_conductores_por_ruta
     (idRuta, idCondutor, incidencias)
VALUES
```

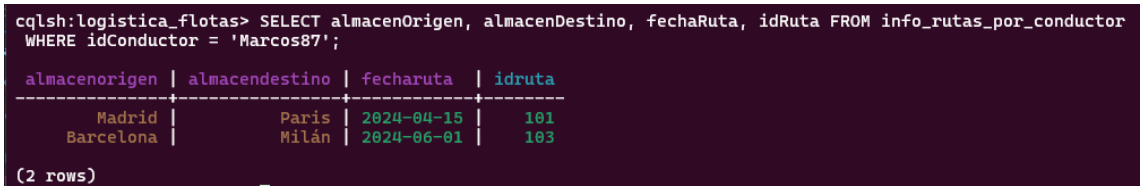
```
        (103, 'Noel99', {'Descanso' : 'Parada obligatoria de
    45 mins'});
```



Apartado 3.

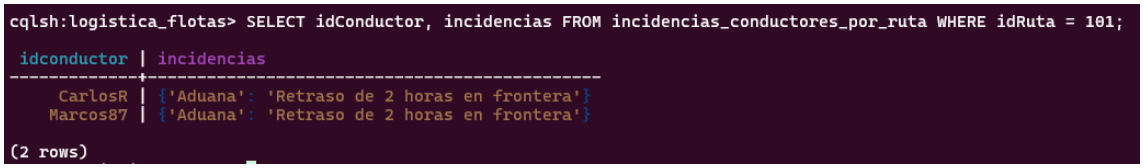
Para la primera petición se debe ejecutar la consulta:

```
SELECT almacenOrigen, almacenDestino, fechaRuta, idRuta
FROM info_rutas_por_conductor
WHERE idConductor = 'Marcos87';
```



Para la segunda petición se debe ejecutar la consulta:

```
SELECT idConductor, incidencias
FROM incidencias_conductores_por_ruta
WHERE idRuta = 101;
```



Para la tercera petición se debe ejecutar la consulta:

```
SELECT almacenOrigen, almacenDestino, horasEstimadas
FROM info_rutas_por_conductor
WHERE idConductor = 'Marcos87' and fechaRuta >= '2024-05-
01'
ALLOW FILTERING;
```

```
cqlsh:logistica_flotas> SELECT almacenOrigen, almacenDestino, horasEstimadas FROM info_rutas_por_conductor WHERE
idConductor = 'Marcos87' and fechaRuta >= '2024-05-01' ALLOW FILTERING;

 almacenorigen | almacendestino | horasestimadas
---------------+----------------+----------------
     Barcelona |          Milán |             12

(1 rows)
```

Y, por último, para la cuarta petición se debe ejecutar la consulta:

```
SELECT incidencias
FROM incidencias_conductores_por_ruta
WHERE idConductor = 'CarlosR' and idRuta = 101;
```

```
cqlsh:logistica_flotas> SELECT incidencias FROM incidencias_conductores_por_ruta WHERE idConductor = 'CarlosR' and
idRuta = 101;

 incidencias
---------------------------------------------
 {'Aduana': 'Retraso de 2 horas en frontera'}

(1 rows)
```

Apartado 4.

Para el apartado 1, la consulta sería

```
SELECT idRuta
FROM info_rutas_por_conductor
ORDER BY fechaRuta DESC;
```

```
cqlsh:logistica_flotas> SELECT idRuta FROM info_rutas_por_conductor ORDER BY fechaRuta DESC;
InvalidRequest: Error from server: code=2200 [Invalid query] message="ORDER BY is only supported when
the partition key is restricted by an EQ or an IN."
cqlsh:logistica_flotas>
```

El error que sale es "`InvalidRequest: Error from server: code=2200 [Invalid query] message="ORDER BY is only supported when the partition key is restricted by an EQ or an IN.""`.

Este error es debido a que Cassandra no tiene un ordenamiento global en una tabla, sino que lo tiene por cada una de las claves de partición. Además, al no especificar una clave de partición, Cassandra debería de comprobar todos los nodos para obtener los registros y realizar la posterior ordenación entre todos ellos.

Para el apartado 2, la consulta sería

```
SELECT idConductor
FROM incidencias_conductores_por_ruta
WHERE idRuta > 101;
```

```
cqlsh:logistica_flotas> SELECT idConductor FROM incidencias_conductores_por_ruta WHERE idRuta > 101;
InvalidRequest: Error from server: code=2200 [Invalid query] message="Cannot execute this query as it might involve data filtering and thus
 may have unpredictable performance. If you want to execute this query despite the performance unpredictability, use ALLOW FILTERING"
cqlsh:logistica_flotas> SELECT idConductor FROM incidencias_conductores_por_ruta WHERE idRuta > 101 ALLOW FILTERING;
```

El error que sale es "`InvalidRequest: Error from server: code=2200 [Invalid query] message="Cannot execute this query as it might involve data filtering and thus may have unpredictable performance. If you want to execute this query despite the performance unpredictability, use ALLOW FILTERING""`.

Este error es debido a que se está indicando un rango para el campo idRuta (101, ∞), lo cual por defecto no puede ser ejecutado, ya que Cassandra usa la clave de partición como un hash para decidir el nodo en el que guardar la información. Debido a esto, Cassandra debe de comprobar todos los nodos para ver qué registro cumple con la condición, reduciendo considerablemente el rendimiento.

Para el último apartado, la consulta es:

```
UPDATE info_rutas_por_conductor
SET fechaRuta = '2024-04-16'
WHERE idConductor = 'Marcos87' AND idRuta = 101;
```

```
cqlsh:logistica_flotas> UPDATE info_rutas_por_conductor SET fechaRuta = '2024-04-16' WHERE idConductor
 = 'Marcos87' and idRuta = 101;
cqlsh:logistica_flotas>
```

En este caso no se produce ningún error, pues se está determinando un registro en concreto de la tabla al indicar la clave primaria al completo.

Apartado 5.

En el primer caso, la consulta sería:

```
SELECT idRuta, idConductor
FROM info_rutas_por_conductor
WHERE fechaRuta = '2024-04-15' ALLOW FILTERING;
```

```
cqlsh:logistica_flotas> SELECT idRuta, idConductor FROM info_rutas_por_conductor WHERE fechaRuta = '20
24-04-15' ALLOW FILTERING;

 idruta | idconductor
--------+-------------
    101 |     CarlosR

(1 rows)
```

De forma normal (sin el ALLOW FILTERING), Cassandra no sabe en qué clúster buscar, pues no está indicando ninguna clave de particionado. Al indicar ALLOW FILTERING se está forzando a buscar en todos los clústeres aquellas instancias que coincidan con la condición indicada.

Con respecto al segundo caso, primero tendríamos que crear el índice:

```
CREATE INDEX indice_fechaRuta
ON info_rutas_por_conductor (fechaRuta);
```

```
cqlsh:logistica_flotas> CREATE INDEX indice_fechaRuta ON info_rutas_por_conductor (fechaRuta);
cqlsh:logistica_flotas>
```

Para posteriormente poder realizar la consulta

```
SELECT idRuta, idConductor
FROM info_rutas_por_conductor
WHERE fechaRuta = '2024-04-15';
```

Que es igual que la primera solo que eliminando `ALLOW FILTERING`. Esto es debido a que el índice permite obtener los datos directamente, aunque en decremento del rendimiento, pues este índice se encuentra en tablas ocultas repartidas entre todos los nodos, de forma que al ejecutar la consulta cada nodo debe obtener los datos de dicha tabla y devolverlos a un nodo que actúa como recolector. Si existen muchos nodos, se puede producir una saturación de la red interna.

En el último caso, la creación de la tabla sería mediante la consulta:

```
CREATE TABLE fecha_rutas_por_conductor (
      fechaRuta date,
      idConductor text,
      idRuta int,
      PRIMARY KEY (fechaRuta, idRuta, idConductor)
);
```

Para insertar el registro del conductor Marcos87 se tiene que realizar la consulta:

```
      INSERT INTO fecha_rutas_por_conductor (fechaRuta,
idConductor, idRuta)
      VALUES ('2024-04-15', 'Marcos87', 101);
```

Y para obtener la información:

```
      SELECT idRuta, idConductor
      FROM fecha_rutas_por_conductor
      WHERE fechaRuta = '2024-04-15';
```

```
cqlsh:logistica_flotas> CREATE TABLE fecha_rutas_por_conductor (
RY KEY (fechaRuta, idRuta, idConductor)
);                    ... fechaRuta date,
                  ... idConductor text,
                  ... idRuta int,
                  ... PRIMARY KEY (fechaRuta, idRuta, idConductor)
                  ... );
cqlsh:logistica_flotas> INSERT INTO fecha_rutas_por_conductor (fechaRuta, idConductor, idRuta)

                  ... VALUES ('2024-04-15', 'Marcos87', 101);
cqlsh:logistica_flotas> SELECT idRuta, idConductor FROM fecha_rutas_por_conductor WHERE fechaR
uta = '2024-04-15';

 idruta | idconductor
--------+-------------
    101 |    Marcos87

(1 rows)
```

Esta es la opción recomendada principalmente por cómo está diseñado Cassandra: orientado a las consultas. Es decir, al crear esta tabla, cuya clave de partición es la fecha de la ruta, cuando se realice una consulta sobre ella, Cassandra sabrá a que nodo dirigirse, pues los registros se almacenarán agrupados por ese campo.

Apartado 6.

Para establecer la consistencia al nivel QUROUM hay que ejecutar la consulta:

```
CONSISTENCY QUORUM;
```

```
cqlsh:logistica_flotas> CONSISTENCY QUORUM;
Consistency level set to QUORUM.
cqlsh:logistica_flotas>
```

Para tirar uno de los nodos habrá que ejecutar, en este caso el 2:

```
sudo docker exec -it cassandra-node2 nodetool drain
```

que sería para conseguir un apagado limpio. Podemos ver que el nodo 1 solo tiene ahora dos conexiones activas, él mismo y el nodo 3:

```
manu@pcmanu:~/GAINE/practica4$ sudo docker exec -it cassandra-node1 nodetool status
Datacenter: datacenter1
=======================
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
--  Address     Load        Tokens  Owns (effective)  Host ID                               Rack
DN  172.20.0.3  80.05 KiB   16      100.0%            5f7d77d5-2cf2-4b50-b414-f67a278f6773  rack1
UN  172.20.0.2  123.46 KiB  16      100.0%            0844a94e-97ff-48d9-90fe-d0280c52c6f3  rack1
UN  172.20.0.4  80.07 KiB   16      100.0%            86f6c830-2933-45db-ab64-292fe28653c5  rack1
```

Si ahora ejecutamos la consulta anterior:

```
SELECT idRuta, idConductor
FROM info_rutas_por_conductor
WHERE fechaRuta = '2024-04-15' ALLOW FILTERING;
```

Observamos que, efectivamente, recibimos respuesta:

```
cqlsh:logistica_flotas> SELECT idRuta, idConductor FROM info_rutas_por_conductor WHERE fechaRuta = '20
24-04-15' ALLOW FILTERING;

 idruta | idconductor
--------+-------------
    101 |     CarlosR

(1 rows)
```

Pues, según la fórmula del quorum, $Q = \left(\frac{N}{2}\right) + 1$, con N el factor de replicación nos da que el quorum es 2, justo el número de nodos que disponemos.

Cuando se tira un segundo nodo, en este caso el 3:

```
sudo docker exec -it cassandra-node3 nodetool drain
```

```
manu@pcmanu:~/GAINE/practica4$ sudo docker exec -it cassandra-node1 nodetool status
Datacenter: datacenter1
=======================
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
--  Address     Load        Tokens  Owns (effective)  Host ID                               Rack
DN  172.20.0.3  80.05 KiB   16      100.0%            5f7d77d5-2cf2-4b50-b414-f67a278f6773  rack1
UN  172.20.0.2  123.46 KiB  16      100.0%            0844a94e-97ff-48d9-90fe-d0280c52c6f3  rack1
DN  172.20.0.4  119.76 KiB  16      100.0%            86f6c830-2933-45db-ab64-292fe28653c5  rack1
```

Cuando ejecutamos la consulta anterior nos sale un error indicando que no se tiene el quorum requerido, pues solo se tiene un nodo disponible:

```
cqlsh:logistica_flotas> SELECT idRuta, idConductor  FROM info_rutas_por_conductor  WHERE fechaRuta = '2024-04-15' ALL
OW FILTERING;
NoHostAvailable: ('Unable to complete the operation against any hosts', {<Host: 127.0.0.1:9042 datacenter1>: Unavaila
ble('Error from server: code=1000 [Unavailable exception] message="Cannot achieve consistency level QUORUM" info={\'c
onsistency\': \'QUORUM\', \'required_replicas\': 2, \'alive_replicas\': 1}')})
```

Conclusiones

Esta práctica, a diferencia de la anterior, me parece que aborda perfectamente los principales aspectos de Cassandra sin ser demasiado densa.

Usar Cassandra me ha ayudado a salir de la mentalidad que se obtiene durante el grado (por lo menos en mi universidad) de modelar las estructuras en función de los datos y, más notablemente, en relación con las relaciones entre ellos. Tener que enforcarme en modelar las estructuras en función de lo que se quiere obtener de ellos creo que me ayuda a afrontar los problemas con un punto de vista adicional, que puede que sea el óptimo y que hasta ahora desconocía.

Durante la práctica me he encontrado con pocos problemas, los únicos han sido en relación con el despliegue con Docker, como el comentado en el principio que tuve que modificar el archivo docker-compose.yml, u otro que me ha ocurrido al intentar tirar los nodos, pues no me quedaba claro del todo qué instrucción debía usar de las diferentes opciones que encontraba y he acabado teniendo que eliminar todos los datos usando el comando docker-compose down -v y volviendo a crear todas las estructuras de nuevo, junto con los datos que contenían.

En resumen, creo que junto con la práctica de Redis, ha sido una práctica dinámica, completa y bastante útil para conocer el funcionamiento de Cassandra.